

AI グラウンディング手法の原理と層構造

— 何が比較可能で、何が合成すべきかを理解する —

各手法の「なぜ効くのか」を原理から解説し、
機能の異なる手法を4層モデルで整理する

2026年3月

目次

1. 旧版の問題: なぜガードレールとオントロジーを比較してはいけないのか
2. 4層モデル: グラウンディングの層構造
3. 第1層 — 知識の構造化(原理と手法比較)
4. 第2層 — 知識の配送(原理と手法比較)
5. 第3層 — モデルへの統合(原理と手法比較)
6. 第4層 — 出力の保証(原理と手法比較)
7. 層間合成パターン: 手法の組み合わせ方
8. 技術原理の類似性マトリクス

1. 旧版の問題：なぜガードレールとオントロジーを比較してはいけないのか

旧版の資料では、オントロジー、RAG、ガードレール、ファインチューニング等を同じ座標系上にプロットし比較していた。しかしこれは根本的に問題がある。これらは同じ機能を果たす代替手段ではなく、異なる機能を担う補完的な層だからだ。

手法	やっていること	機能カテゴリ
オントロジー	ドメインの概念・関係を定義する	知識の構造化
RAG	外部テキストを検索して注入する	知識の配送
ファインチューニング	モデルの重みを更新する	モデルへの統合
ガードレール	出力を検証・フィルタリングする	出力の保証

オントロジーとRAGを「どちらが優れているか」と比較することは、建物の設計図と配送トラックを「どちらが優れているか」と比較するのと同じだ。設計図（構造化）は建物の形を定義し、配送トラック（RAG）は建材を届ける。それぞれ異なる問いに答えている。

比較が意味を持つのは、同じ問いに対する異なる答えの間だけである。例:「知識をどう構造化するか」という問いに対して、オントロジー vs ナレッジグラフ vs タクソノミーの比較は有意義だ。

比較	意味があるか	理由
オントロジー vs ナレッジグラフ	意味がある	同じ問い「知識をどう構造化するか」への異なる答え
Naive RAG vs Advanced RAG	意味がある	同じ問い「知識をどう届けるか」への異なる答え
ファインチューニング vs プロンプティング	意味がある	同じ問い「知識をどうモデルに入れるか」への異なる答え
オントロジー vs ガードレール	意味がない	答えている問いが異なる（構造化 vs 出力制御）
RAG vs ファインチューニング	条件付き	「知識をモデルに入れる」意味では比較可能だが層が異なる

2. 4層モデル：グラウンディングの層構造

グラウンディング手法を機能に基づき4つの層に整理する。各層は異なる問いに答えており、同一層内の手法同士が比較対象、異なる層の手法同士が合成対象となる。

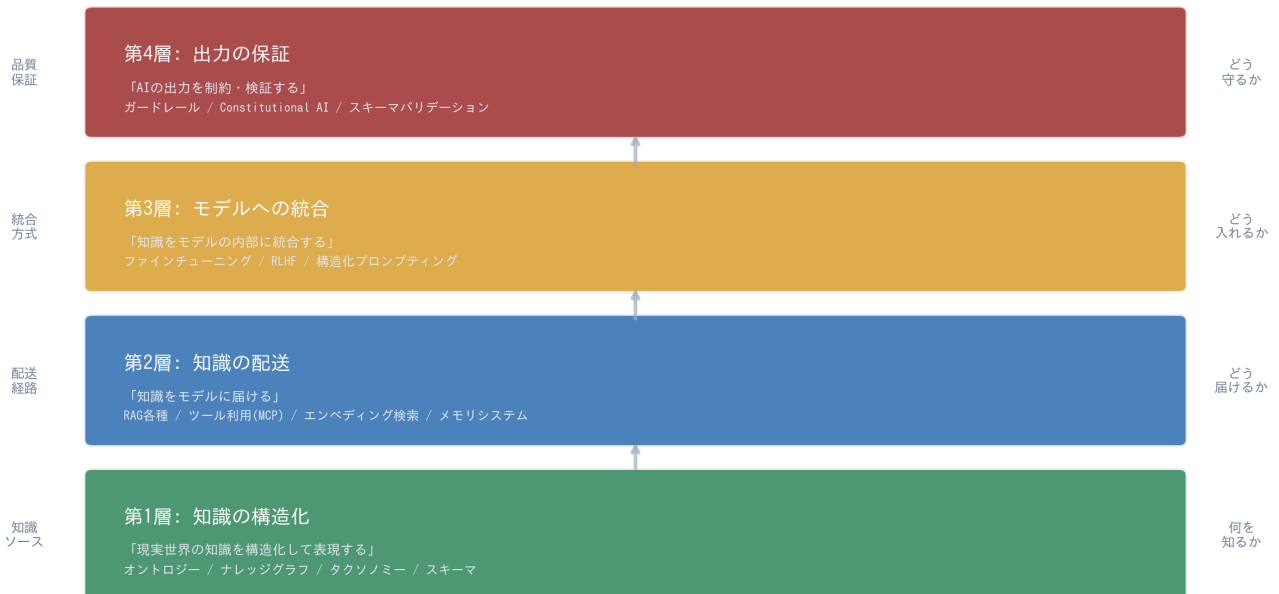


図1: グラウンディングの4層モデル — 各層は異なる問いに答える

各層の定義

第1層：知識の構造化

「現実世界の知識をどう表現するか」に答える層。

知識を人間とAIの両方が理解できる構造化に変換する。表現力が高いほど推論が可能になるが、構築・維持のコストが上がる。

手法: オントロジー、ナレッジグラフ、タクソノミー、スキーマ、平ドキュメント

第2層：知識の配送

「構造化された知識をどうモデルに届けるか」に答える層。

第1層で構造化された知識（またはそのまま）を検索・取得し、モデルが利用できる形で配送する。鮮度と精度のバランスが核心。

手法: RAG各種、エンベディング検索、ツール利用/MCP、メモリシステム

第3層：モデルへの統合

「知識をどうモデルの内部に統合するか」に答える層。

コンテキスト注入（一時的）とパラメータ更新（永続的）の2系統がある。柔軟性と永続性のトレードオフが核心。

手法: ファインチューニング/RLHF、構造化プロンプティング、マルチモーダル入力

第4層：出力の保証

「AIの出力をどう制約・検証するか」に答える層。

グラウンディングの最終防衛線。出力が事実・ルール・安全性基準に適合しているかを検証する。これは知識の提供ではなく品質保証。

手法: ガードレール/Constitutional AI、スキーマバリデーション、形式論理/ルールエンジン

3. 第1層 — 知識の構造化：原理と手法比較

この層の手法は「現実世界の構造を機械可読な形式で記述する」という共通原理を持つ。違いは表現力の深さにある。

平ドキュメント (Markdown/PDF)

何をするか: 非構造化テキストをそのまま知識源として使う。

なぜ効くのか(原理): 自然言語は人間にとって最も表現力が高い形式である。LLMは自然言語の理解に長けているため、構造化せずともある程度の知識伝達が可能。

限界・トレードオフ: 構造がないため、同じ文書でも解釈にブレが生じる。大規模になると検索精度が落ちる。CLAUDE.md等のシステムプロンプトは実質このアプローチ。

タクソノミー / 統制語彙

何をするか: 概念を階層的な「is-a」関係で分類する。

なぜ効くのか(原理): 人間の認知は分類(カテゴライゼーション)を基盤とする。AIも語彙が統制されることで同義語・多義語の混乱がなくなり、検索精度と推論の一貫性が向上する。

限界・トレードオフ: 表現できるのは上下関係のみ。「AはBの一種」以外の関係(原因、構成要素等)は表現不可。

スキーマ / データモデル

何をするか: JSON Schema、SQL DDL等でデータの構造・型・制約を定義する。

なぜ効くのか(原理): データの「形」を厳密に定義することで、AIの入出力を予測可能にする。構造化出力(Structured Output)の基盤であり、システム間の契約(API Contract)として機能する。

限界・トレードオフ: 個々のデータの形は定義できるが、概念間の意味的關係は表現できない。「顧客」と「注文」の関係はFK制約で表現するが、意味的豊かさはない。

ナレッジグラフ

何をするか: エンティティ(ノード)と関係(エッジ)のグラフ構造で知識を表現する。

なぜ効くのか(原理): グラフ構造は多対多の関係を自然に表現でき、「AがBの原因で、BがCを構成する」のようなマルチホップ推論が可能。トリプル(主語-述語-目的語)の集積が知識の網を形成する。

限界・トレードオフ: 任意の関係を定義できるが、関係の意味は暗黙的。「AはBに関連する」の「関連」の厳密な意味は保証されない。

OWLオントロジー

何をするか: 記述論理(Description Logic)に基づき、概念・関係・公理を厳密に定義する。

なぜ効くのか(原理): 形式論理に基づくため推論器(Reasoner)による自動導出が可能。例:「哺乳類は脊椎動物である」「クジラは哺乳類である」→「クジラは脊椎動物である」を自動推論。矛盾検出・分類の自動化も可能。

限界・トレードオフ:

最も表現力が高いが、構築にOWL/RDFの専門知識が必須。ドメインエキスパートとオントロジストの協業が必要で、コストが最大。

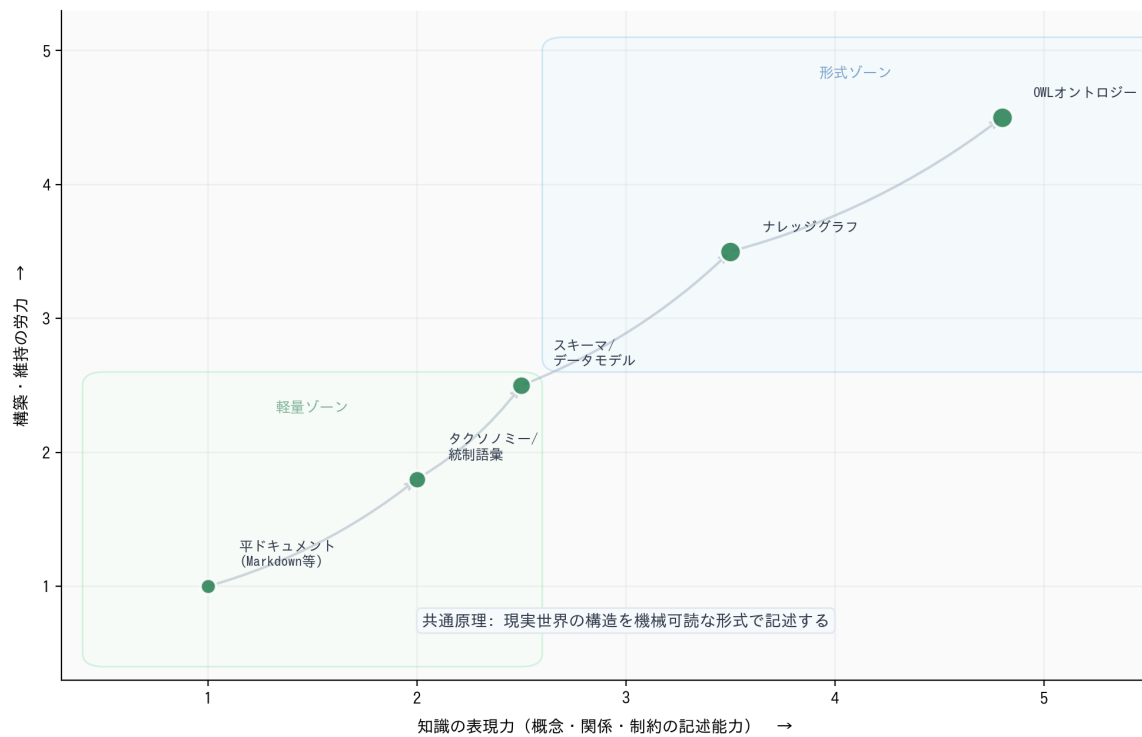


図2: 第1層の手法比較 — 表現力と構築コストのトレードオフ (矢印は進化方向)

類似性の核心: すべて「現実をモデル化する」が、モデルの精密さが異なる。平ドキュメントは自然言語という曖昧な表現、タクソノミーは階層のみ、スキーマは型と制約、ナレッジグラフは関係のネットワーク、オントロジーは論理的公理。表現力の階段を登るほど機械推論が可能になるが、人間の負荷も上がる。

4. 第2層 — 知識の配送：原理と手法比較

この層の手法は「外部の知識をモデルのコンテキストに配送する」という共通原理を持つ。違いは配送のメカニズムにある。

Naive RAG

何をするか: クエリで文書を検索し、上位結果をそのままプロンプトに追加する。

なぜ効くのか(原理): 「最も関連性の高い情報を見つけてモデルに渡す」という最もシンプルな実装。ベクトル類似度がクエリと文書の意味的近さの近似として機能する。

限界・トレードオフ: チャンク分割の品質に依存。文脈の喪失、無関係な情報の混入が頻発。

Advanced RAG (ハイブリッド検索)

何をするか: クエリ変換、リランキング、ハイブリッド検索(密+疎)を組み合わせる。

なぜ効くのか(原理): Naive RAGの各ステップを個別に最適化する。クエリ拡張で意図を明確化し、リランカーで精度を向上、ハイブリッド検索でキーワード一致と意味検索を両立する。

限界・トレードオフ: パイプラインが複雑化し、レイテンシが増大。各ステップのチューニングが必要。

Graph RAG

何をするか: ナレッジグラフの構造を利用して、関連エンティティを辿りながら情報を収集する。

なぜ効くのか(原理): グラフ走査により「AがBに影響し、BがCを引き起こす」のような多段階の推論チェーンを構築できる。テキスト検索では見つけられない構造的な関連性を発見する。

限界・トレードオフ: 第1層のナレッジグラフ構築が前提。グラフの品質が直接精度に影響。

エンベディング検索 (ベクトル検索)

何をするか: テキストを密ベクトルに変換し、ベクトル空間上の近傍検索で類似内容を取得する。

なぜ効くのか(原理): 意味的に類似したテキストはベクトル空間上で近い位置に配置される(分布仮説)。これにより、キーワードが一致しなくても意味的に関連する情報を発見できる。

限界・トレードオフ: 「意味が近い」と「回答に役立つ」は必ずしも一致しない。

ツール利用 / MCP

何をするか: LLMがAPI呼び出しを介して外部システムにリアルタイムアクセスする。

なぜ効くのか(原理): モデルは「どのツールを、どの引数で呼ぶか」を判断し、結果をコンテキストに統合する。MCPにより接続の標準化が実現。検索ではなく実行(API呼び出し、計算、データ取得)が本質。

限界・トレードオフ: ツールの信頼性に依存。API呼び出しのレイテンシとコスト。安全性の確保が課題。

メモリシステム (MemGPT/Letta等)

何をするか: LLMの固定コンテキストウィンドウを超えた持続的記憶を提供する。

なぜ効くのか(原理): OSの仮想メモリに着想を得た設計。メインメモリ(コンテキストウィンドウ)とアーカイブ(外部ストレージ)をLLM自身が管理し、必要な記憶をページイン/アウトする。

限界・トレードオフ: 何を記憶し何を忘れるかの判断をLLMに委ねるリスク。実装の複雑さ。

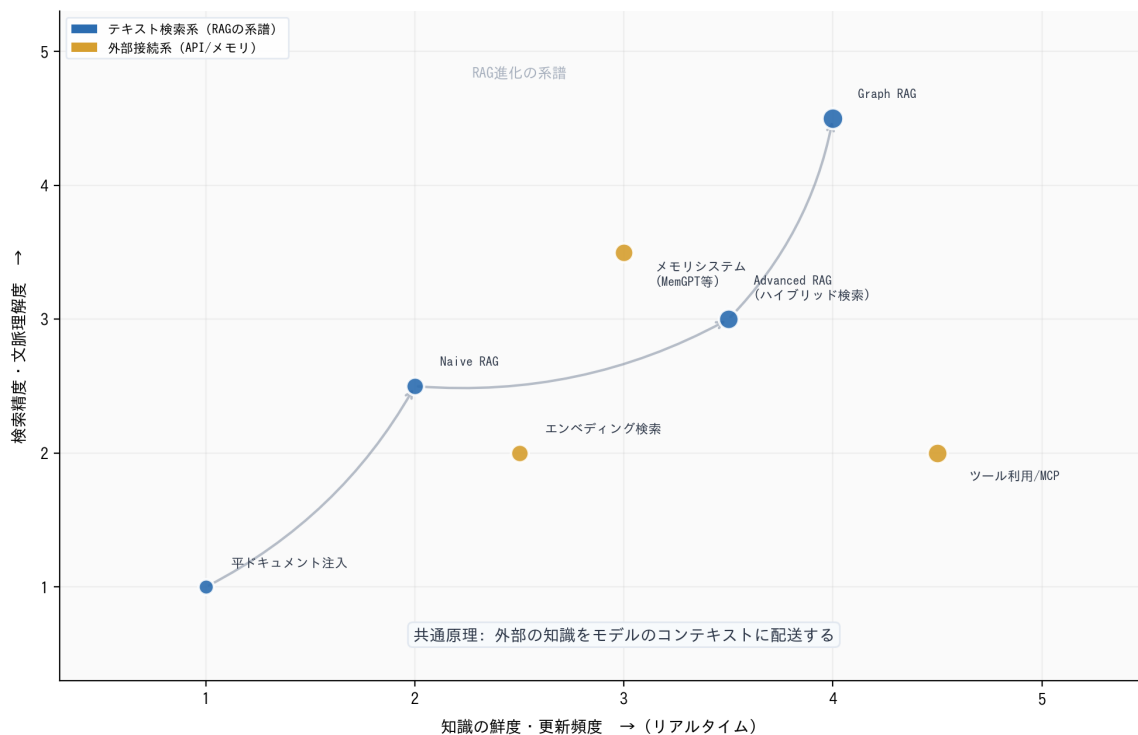


図3: 第2層の手法比較 — 鮮度×精度 (矢印はRAG進化の系譜)

類似性の核心: すべて「外部知識をモデルに届ける」が、配送メカニズムが異なる。RAG系は「事前に蓄えた文書を検索」、ツール利用は「リアルタイムにAPIを呼ぶ」、メモリは「過去の対話を呼び戻す」。検索 vs 実行 vs 記憶想起という3つのアプローチ。

5. 第3層 — モデルへの統合：原理と手法比較

この層の手法は「知識をモデルの内部に統合する」という共通原理を持つ。核心的な違いは一時的（コンテキスト注入）か永続的（パラメータ更新）かにある。

構造化プロンプティング（CoT、Few-shot等）

何をするか：プロンプト設計により、モデルの推論パターンを誘導する。

なぜ効くのか（原理）：LLMはin-context learningの能力を持つ。適切な例示（Few-shot）や推論ステップの明示（Chain-of-Thought）により、パラメータを変更せずにモデルの振る舞いを変えられる。知識というよりも「考え方の型」を注入する。

限界・トレードオフ：一時的。コンテキストウィンドウの消費。プロンプトの品質に強く依存。モデルが変わると再設計が必要。

ファインチューニング / RLHF

何をするか：ドメイン固有のデータでモデルのパラメータ（重み）を更新する。

なぜ効くのか（原理）：勾配降下法によりモデルの内部表現を変更し、特定タスク・ドメインへの適応を実現する。RLHFは人間のフィードバックを報酬関数に変換し、モデルの行動選好を調整する。知識がパラメータに「焼き込まれる」。

限界・トレードオフ：

永続的だが更新が困難（再訓練が必要）。大量の学習データと計算資源が必要。新しい知識との矛盾（catastrophic forgetting）。事実の埋め込みよりもスタイル・トーン・行動パターンの制御に向く。

マルチモーダル入力

何をするか：画像・音声・動画等の非テキストデータをモデルに入力する。

なぜ効くのか（原理）：人間の認知は多感覚統合で成り立つ。テキストだけでは伝えにくい空間的・視覚的情報を直接入力することで、モデルの理解を物理世界に固定する。設計図、写真、グラフなどが典型。

限界・トレードオフ：一時的（コンテキストウィンドウ内）。モダリティ間の解釈ギャップ。テキストより大きなトークン消費。

統合方式の比較：一時的 vs 永続的

観点	一時的統合 （プロンプト/コンテキスト）	永続的統合 （ファインチューニング）
原理	in-context learningで推論時に知識を提供	勾配降下法でパラメータに知識を焼き込む
永続性	セッション終了で消失	モデルに永続的に残る
更新容易性	プロンプト編集で即反映	再訓練が必要（時間・コスト大）
適する知識	頻繁に変化する事実・最新情報	安定したスタイル・行動パターン
リスク	コンテキストウィンドウの圧迫	catastrophic forgetting

類似性の核心：すべて「モデルに知識を持たせる」が、保持期間と更新方法が根本的に異なる。多くの実用システムではファインチューニングでスタイルを、プロンプティングで事実を、マルチモーダルで視覚情報を統合する多層利用が最適解となる。

6. 第4層 — 出力の保証：原理と手法比較

この層は他の3層とは本質的に異なる。他の層が知識をモデルに入れるのに対し、この層はモデルの出力を検証・制約する。グラウンディングの「最終防衛線」。

ガードレール / Constitutional AI

何をするか: AIの出力をルールやポリシーに基づきリアルタイムでフィルタリング・修正する。

なぜ効くのか(原理): LLMの出力は確率的であり、常に正確とは限らない。事後的にルールベースの検証を行うことで、ハルシネーション、有害表現、ポリシー違反を検出・阻止する。Constitutional AIは「自分で自分を批判・修正する」メタ認知的アプローチ。

限界・トレードオフ:

検出はできるが修正は限定的。ルールの網羅性に依存。レイテンシの増加。正当な出力を誤ってブロックするリスク(偽陽性)。

スキーマバリデーション (構造化出力)

何をするか: AIの出力をJSON Schema等の型定義に適合させる。

なぜ効くのか(原理): 出力の「形」を制約することで、下流システムとの統合を保証する。Structured Outputにより、APIレスポンスとして確実にパース可能な形式を強制する。

限界・トレードオフ: 構造の正しさは保証するが、内容の正しさは保証しない。「正しいJSON形式だが事実と異なる」出力は防げない。

形式論理 / ルールエンジン (出力検証として)

何をするか: AIの出力を述語論理やIF-THENルールで検証する。

なぜ効くのか(原理): 形式論理に基づく検証は、矛盾の検出と整合性の保証が可能。例:「年齢がマイナス」「開始日が終了日より後」等の論理矛盾を自動検出する。ドメイン固有の業務ルールの強制にも使える。

限界・トレードオフ:

ルールの定義・維持に専門知識が必要。定量的な判断(「正確な数値か」)は可能だが、定性的な判断(「適切な表現か」)は困難。

なぜ「グラウンディング」に含めるのか: ガードレール等は知識の提供ではないが、「AIの出力を現実根拠づける」という広義のグラウンディングの最終段階を担う。知識を入れるだけでは不十分で、出力が知識と整合しているかの検証が不可欠。ただし、他の層の手法と同列に比較するのは誤りであり、補完的な役割として理解すべきである。

7. 層間合成パターン：手法の組み合わせ方

実用的なグラウンディングは複数の層の手法を合成して構築する。以下に典型的な合成パターンを示す。

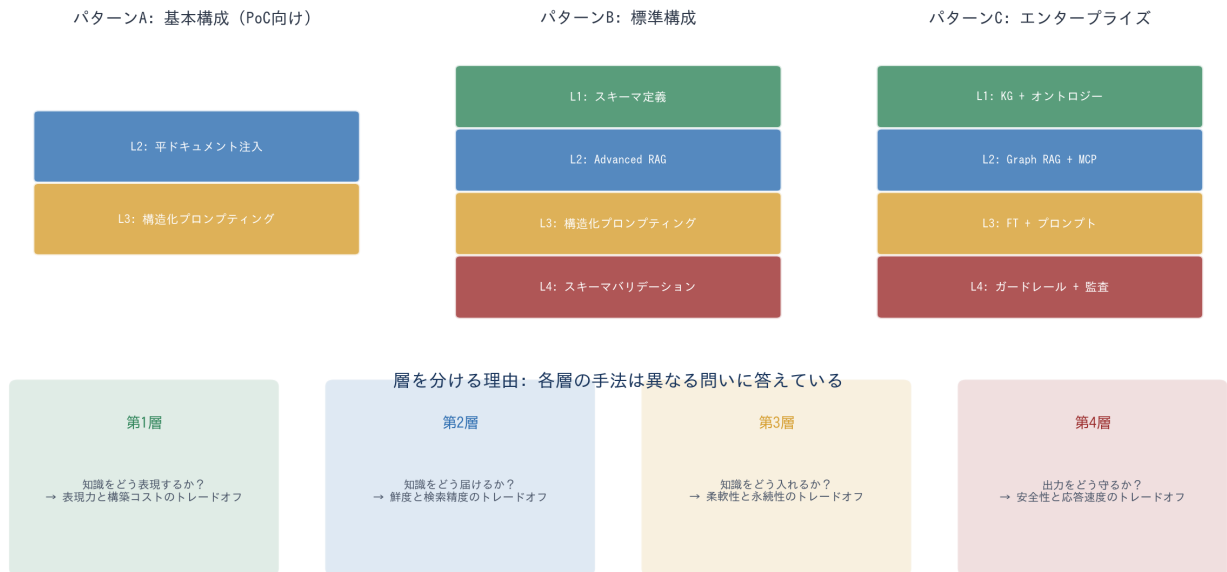


図4: 層間合成パターン — 用途に応じた手法の積み方

パターンA: 基本構成 (PoC・個人利用)

構成: L2: 平ドキュメント注入 → L3: 構造化プロンプティング

CLAUDE.mdにドメイン知識を書き、プロンプトで推論を誘導する最小構成。インフラ不要で即日開始可能。小規模では十分だがスケールしない。

パターンB: 標準構成 (プロダクト)

構成: L1: スキーマ定義 → L2: Advanced RAG → L3: プロンプティング → L4: スキーマ検証

ドキュメントをチャンク化・検索し、構造化出力で応答する標準的なRAGアプリ。スキーマが入出力の「契約」として機能し、第1層と第4層で品質を挟み込む。

パターンC: エンタープライズ構成

構成: L1: KG+オントロジー → L2: Graph RAG+MCP → L3: FT+プロンプト → L4: ガードレール+監査

4層すべてを実装した最も堅牢な構成。金融・医療・法律等のミッションクリティカルな領域で必要。導入・維持のコストは最大。

パターンD: リアルタイム構成 (IoT/Physical AI)

構成: L1: デジタルツイン → L2: MCP+センサー → L3: マルチモーダル → L4: 物理法則検証

物理世界のリアルタイムデータに基づきAIが判断する構成。製造制御、自動運転、ロボティクス等に適用。

8. 技術原理の類似性マトリクス

各手法の原理を5つの観点で整理し、どの手法が「似ている」のかを明確にする。同じ層内の手法は似ており、異なる層の手法は補完的であることが読み取れる。

手法	層	核心原理	入力	出力	持続性
オントロジー	1	論理的公理で概念を定義	ドメイン知識	形式的定義	永続
ナレッジグラフ	1	トリプルで関係を記述	エンティティ	グラフ構造	永続
タクソノミー	1	階層分類で概念を整理	用語・概念	階層木	永続
スキーマ	1	型と制約でデータの形を定義	データ構造	型定義	永続
Naive RAG	2	ベクトル類似度で検索	クエリ	関連テキスト	一時
Advanced RAG	2	多段パイプラインで精度向上	クエリ	精選テキスト	一時
Graph RAG	2	グラフ走査で関連を辿る	クエリ+グラフ	推論チェーン	一時
ツール利用/MCP	2	API呼び出しでデータ取得	ツール定義	API応答	一時
メモリシステム	2	仮想メモリで記憶を管理	対話履歴	想起された記憶	半永続
プロンプティング	3	in-context learningを活用	テンプレート	推論誘導	一時
ファインチューニング	3	勾配降下法で重みを更新	学習データ	パラメータ変更	永続
マルチモーダル	3	多感覚統合で理解を補強	画像/音声等	統合表現	一時
ガードレール	4	ルールベースの事後検証	AI出力	検証済み出力	永続ルール
スキーマ検証	4	型適合の強制	AI出力	構造化出力	永続定義
ルールエンジン	4	述語論理で矛盾検出	AI出力	検証結果	永続ルール

結語: グラウンディング手法の選定は「どの1つを選ぶか」ではなく、「4つの問い(構造化・配送・統合・保証)にそれぞれどう答えるか」を設計することである。同じ層内では要件に応じて最適な手法を選択し、異なる層の手法は積み重ねて合成する。これが本資料の最も重要なメッセージである。